

```

import numpy as np

# Input data
hours = np.array([1,2,3,4,5,6,7,8,9,10])
scores = np.array([60,70,80,90,100,110,120,130,140,150])

# Linear Regression tries to find the BEST FIT LINE
# Equation of line:
# score = m * hours + c
#
# m = slope of the line
# c = intercept of the line

# Manual understanding of slope:
# slope = change in y / change in x
# slope = (150 - 60) / (10 - 1)
# slope = 90 / 9
# slope = 10

# Intercept:
# c = y - mx
# c = 60 - (10 * 1)
# c = 50

# Final best fit equation:
# predicted_score = 10 * hours + 50

# Predict score for 5 study hours
predicted_score = 10 * 5 + 50

print("Best Fit Line Equation:")
print("score = 10 * hours + 50")

print("\nFor 5 hours → predicted score:", predicted_score)

```

For 5 hours → predicted score: 68.4

Each value in hours represents study hours.

Each value in scores represents exam marks.

Linear Regression finds the best fit line that represents the relationship between these two variables.

The slope (m) tells how much the score increases when study hours increase by 1.

The intercept (c) is the starting point of the line.

```

import numpy as np
import matplotlib.pyplot as plt

# Original data
hours = np.array([1,2,3,4,5,6,7,8,9,10])
scores = np.array([60,70,80,90,100,110,120,130,140,150])

# Best Fit Line Equation
# y = 10x + 50

predicted_scores = 10.41 * hours + 48.52

# Create graph
plt.figure(figsize=(8,6))

# Actual data points
plt.scatter(hours, scores, label="Actual Data", marker="^", color="red")

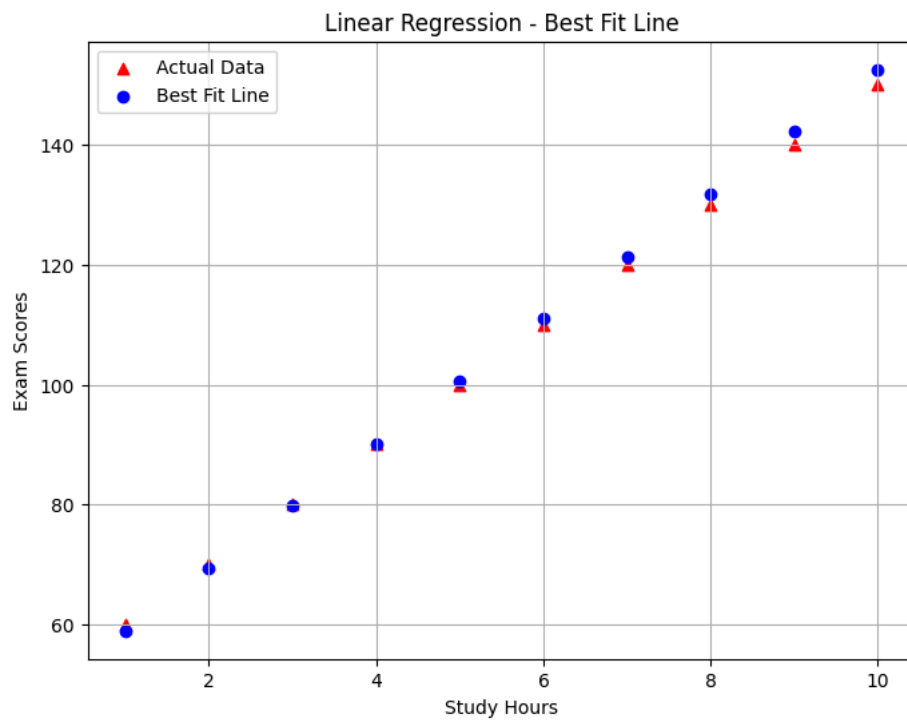
# Best fit regression line
plt.scatter(hours, predicted_scores, label="Best Fit Line", color="blue")

# Labels and title
plt.xlabel("Study Hours")
plt.ylabel("Exam Scores")
plt.title("Linear Regression - Best Fit Line")

# Show legend
plt.legend()

# Show graph
plt.grid(True)
plt.show()

```



```
import numpy as np

# -----
# Dataset
# -----
# x = study hours
# y = actual scores

x = np.array([1,2,3,4,5], dtype=float)
y = np.array([60,70,80,90,100], dtype=float)

# -----
# Initial values of m and c
# The model initially knows nothing.
# -----
m = 0
c = 0

# Learning rate
learning_rate = 0.01

# Number of iterations
epochs = 1000

# Total number of data points
n = len(x)

# -----
# Gradient Descent
# -----

for i in range(epochs):

    # Predicted values
    y_pred = m * x + c

    # Error
    error = y - y_pred

    # Mean Squared Error (MSE)
    mse = np.mean(error ** 2)

    # Gradients
    dm = (-2/n) * np.sum(x * error) #How much the slope (m) should change
    dc = (-2/n) * np.sum(error) #How much the intercept (c) should change

    # Update m and c
    m = m - learning_rate * dm
    c = c - learning_rate * dc

    # Print progress every 100 iterations
    if i % 100 == 0:
```

```

print(f"Epoch {i}")
print(f"MSE = {mse:.2f}")
print(f"m = {m:.2f}, c = {c:.2f}")
print("-----")

# Final equation
print("\nFinal Best Fit Line:")
print(f"y = {m:.2f}x + {c:.2f}")

```

```

Epoch 0
MSE = 6600.00
m = 5.20, c = 1.60
-----
Epoch 100
MSE = 177.90
m = 18.63, c = 18.84
-----
Epoch 200
MSE = 90.37
m = 16.15, c = 27.79
-----
Epoch 300
MSE = 45.90
m = 14.38, c = 34.17
-----
Epoch 400
MSE = 23.32
m = 13.12, c = 38.72
-----
Epoch 500
MSE = 11.84
m = 12.23, c = 41.96
-----
Epoch 600
MSE = 6.02
m = 11.59, c = 44.27
-----
Epoch 700
MSE = 3.06
m = 11.13, c = 45.92
-----
Epoch 800
MSE = 1.55
m = 10.81, c = 47.09
-----
Epoch 900
MSE = 0.79
m = 10.57, c = 47.93
-----

Final Best Fit Line:
y = 10.41x + 48.52

```

### Building Models with sklearn

```

from sklearn.model_selection import train_test_split
import numpy as np

# Example: salary prediction from experience
experience = np.array([1,2,3,4,5,6,7,8,9,10,11,12,13,14,15])
salary = np.array([35,38,42,48,55,60,65,70,76,82,87,92,96,100,105])

X = experience.reshape(-1, 1) # sklearn expects 2D array for features
y = salary

X_train, X_test, y_train, y_test = train_test_split(
    X, y,

```

```
test_size=0.2, # 20% for testing
random_state=42 # ensures same split every run
)
```

```
print(f"Training samples : {len(X_train)}") # 12
print(f"Testing samples : {len(X_test)}") # 3
```

```
Training samples : 12
Testing samples : 3
```

### Fitting and Predicting

```
from sklearn.linear_model import LinearRegression
# 1. Create the model object
model = LinearRegression()

# 2. Train (fit) on training data
model.fit(X_train, y_train)

# 3. Inspect what the model learned
print(f"Slope (coefficient) : {model.coef_[0]:.2f}") # slope=coef
print(f"Intercept : {model.intercept_: .2f}")

# 4. Predict on TEST data (unseen!)
y_pred = model.predict(X_test)

# 5. See predictions vs actual
for actual, predicted in zip(y_test, y_pred):
    print(f"Actual: {actual:>5} | Predicted: {predicted:>7.1f} | "
          f"Error: {predicted - actual:+.1f}")
```

```
Slope (coefficient) : 5.22
Intercept : 27.94
Actual:    82 | Predicted:    80.2 | Error: -1.8
Actual:    92 | Predicted:    90.6 | Error: -1.4
Actual:    35 | Predicted:    33.2 | Error: -1.8
```

| Metric                     | What it tells                | Best for               |
|----------------------------|------------------------------|------------------------|
| <b>R<sup>2</sup> Score</b> | How well model explains data | Model performance      |
| <b>MAE</b>                 | Average error                | Simple interpretation  |
| <b>RMSE</b>                | Penalizes large errors       | Detecting big mistakes |

### calculating all three metrics

```
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
import numpy as np

y_actual = np.array([500, 300, 700, 450, 600])
y_pred = np.array([480, 320, 690, 460, 590])

#How much of the variation in data is explained by the model
r2 = r2_score(y_actual, y_pred)

#Average of absolute errors (ignores + / - signs)
mae = mean_absolute_error(y_actual, y_pred)

#RMSE (Root Mean Squared Error)
#Average error but gives more weight to big mistakes
rmse = np.sqrt(mean_squared_error(y_actual, y_pred))
```

```
print(f"R² Score : {r2:.4f} → model explains {r2*100:.1f}% of variance")
print(f"MAE : {mae:.1f} → avg error ± {mae:.0f} units")
print(f"RMSE : {rmse:.1f} → penalised avg error {rmse:.0f} units")
# Interpretation:
# R² = 0.95 means 95% of the price variation is explained by our features
# MAE = 15000 means we're off by ~$15,000 on average
# RMSE > MAE means there are some larger individual errors
```

```
R² Score : 0.9880 → model explains 98.8% of variance
MAE : 14.0 → avg error ± 14 units
RMSE : 14.8 → penalised avg error 15 units
```

| Metric          | What it tells                 | Best use                   |
|-----------------|-------------------------------|----------------------------|
| <b>R² Score</b> | How good the model is overall | Model quality              |
| <b>MAE</b>      | Average error                 | Easy interpretation        |
| <b>RMSE</b>     | Penalizes big errors          | When large mistakes matter |

### Multiple Features

– Predicting Salary Real problems have multiple input features. Linear regression handles this naturally: it learns one weight (coefficient) per feature

What are Multiple Features?

in real life, salary depends on many things:

Experience

Education level

Skills

City

Company

Certifications

**salary =  $w_1 \cdot \text{experience} + w_2 \cdot \text{education} + w_3 \cdot \text{age} + b$**   
*Each feature gets its own weight (importance)*

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import r2_score, mean_absolute_error
import numpy as np

# Create a realistic salary dataset
np.random.seed(42)
n = 100
data = pd.DataFrame({
    'experience': np.random.randint(0, 20, n), # 0-20 years
    'education': np.random.randint(12, 20, n), # 12-20 years schooling
    'age': np.random.randint(22, 55, n), # 22-55 years
})

# Generate salary with some noise
data['salary'] = (
    data['experience'] * 4500 +
    data['education'] * 2000 +
```

```

data['age'] * 300 +
np.random.normal(0, 3000, n) +
10000
)

# Prepare features and target
X = data[['experience', 'education', 'age']]
y = data['salary']
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
model = LinearRegression()
model.fit(X_train, y_train)

# Print learned weights
for feat, coef in zip(X.columns, model.coef_):
    print(f" {feat:12s}: ${coef:,.0f} per unit increase")

print(f" Base salary (intercept): ${model.intercept_:,.0f}")

y_pred = model.predict(X_test)
print(f"\nR² : {r2_score(y_test, y_pred):.3f}")
print(f"MAE : ${mean_absolute_error(y_test, y_pred):,.0f}")

```

```

experience : $4,534 per unit increase
education  : $2,203 per unit increase
age        : $266 per unit increase
Base salary (intercept): $7,694

```

```

R² : 0.988
MAE : $2,219

```

## LAB

### Step-1: create the data set

#### Generate Realistic House Price Data

```

import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error
np.random.seed(42)
n = 200
locations = ['Urban', 'Suburban', 'Rural']
df = pd.DataFrame({
    'area_sqft': np.random.randint(500, 4000, n),
    'num_rooms': np.random.randint(1, 7, n),
    'location': np.random.choice(locations, n),
    'age_years': np.random.randint(0, 40, n),
})

# Location multipliers
loc_map = {'Urban': 1.5, 'Suburban': 1.0, 'Rural': 0.7}#expensiv medium cheap
noise = np.random.normal(0, 20000, n)
df['price'] = (
    df['area_sqft'] * 150 +
    df['num_rooms'] * 25000 -
    df['age_years'] * 2000 +
    df['location'].map(loc_map) * 50000 +
    noise
).astype(int)

print(df.head(8).to_string(index=False))

```

```
print(f"\nDataset shape : {df.shape}")
print(df.describe()[['area_sqft', 'num_rooms', 'age_years', 'price']].round(0))
```

| area_sqft | num_rooms | location | age_years | price  |
|-----------|-----------|----------|-----------|--------|
| 3674      | 4         | Rural    | 20        | 646067 |
| 1360      | 4         | Urban    | 35        | 300479 |
| 1794      | 6         | Urban    | 9         | 501609 |
| 1630      | 6         | Rural    | 36        | 355668 |
| 1595      | 3         | Urban    | 8         | 391991 |
| 3592      | 2         | Rural    | 23        | 566413 |
| 2138      | 4         | Suburban | 34        | 410974 |
| 2669      | 1         | Urban    | 34        | 459191 |

Dataset shape : (200, 5)

|       | area_sqft | num_rooms | age_years | price    |
|-------|-----------|-----------|-----------|----------|
| count | 200.0     | 200.0     | 200.0     | 200.0    |
| mean  | 2334.0    | 3.0       | 20.0      | 451548.0 |
| std   | 992.0     | 2.0       | 11.0      | 154871.0 |
| min   | 521.0     | 1.0       | 0.0       | 86373.0  |
| 25%   | 1519.0    | 2.0       | 10.0      | 326188.0 |
| 50%   | 2372.0    | 3.0       | 20.0      | 458282.0 |
| 75%   | 3230.0    | 5.0       | 30.0      | 575910.0 |
| max   | 3999.0    | 6.0       | 39.0      | 781261.0 |

## Step 2 — Encode & Prepare Features

```
# Encode location: Urban=2, Suburban=1, Rural=0
#Creates a tool that converts text → numbers
le = LabelEncoder()
#transform Converts them into numbers automatically
df['location_enc'] = le.fit_transform(df['location'])
# Select features and target
features = ['area_sqft', 'num_rooms', 'age_years', 'location_enc']
X = df[features]
y = df['price']
# Peek at encoded data
print(df[['location', 'location_enc']].drop_duplicates().sort_values('location_enc'))
print(f"\nFeature matrix shape : {X.shape}")
print(f"Target vector shape : {y.shape}")
```

|   | location | location_enc |
|---|----------|--------------|
| 0 | Rural    | 0            |
| 6 | Suburban | 1            |
| 1 | Urban    | 2            |

Feature matrix shape : (200, 4)  
Target vector shape : (200,)

## Step 3 — Train / Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
print(f"Training samples : {len(X_train)} ({len(X_train)/len(X)*100:.0f}%)")
print(f"Testing samples : {len(X_test)} ({len(X_test)/len(X)*100:.0f}%)")
print(f"Feature names : {features}")
```

Training samples : 160 (80%)  
Testing samples : 40 (20%)  
Feature names : ['area\_sqft', 'num\_rooms', 'age\_years', 'location\_enc']

## Step 4 — Train the Model

```
model = LinearRegression()
model.fit(X_train, y_train)
print("Learned Coefficients:")
print("-" * 45)
for feat, coef in zip(features, model.coef_):
    direction = "↑" if coef > 0 else "↓"
    print(f" {feat:15s}: Rs {coef:>10,.0f} {direction}")
print(f" {'intercept':15s}: Rs {model.intercept_:>10,.0f}")
print("-" * 45)
# Interpretation:
# area_sqft → every extra sq ft adds ~Rs 150 to price
# num_rooms → every extra room adds ~Rs 25,000
# age_years → every year older reduces by ~Rs 2,000
```

Learned Coefficients:

| area_sqft | : Rs | 151 ↑    |
|-----------|------|----------|
| num_rooms | : Rs | 23,169 ↑ |

```

age_years      : Rs      -2,262 ↓
location_enc   : Rs      18,114 ↑
intercept      : Rs      46,789
-----

```

## Step 5 — Evaluate

```

y_pred = model.predict(X_test)
r2 = r2_score(y_test, y_pred)
mae = mean_absolute_error(y_test, y_pred)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
print("=" * 40)
print(" MODEL EVALUATION REPORT")
print("=" * 40)
print(f"R² Score : {r2:.4f} {'✓ Excellent' if r2>0.9 else '✓ Good' if r2>0.8 else '■ Needs work'}")
print(f"MAE : Rs {mae:>10,.0f}")
print(f"RMSE : Rs {rmse:>10,.0f}")
print("=" * 40)
# Show first 8 predictions vs actuals
print("\nSample Predictions:")
print(f"{'Actual':>10} | {'Predicted':>10} | {'Error':>10}")
print("-" * 36)
for a, p in zip(y_test[:8], y_pred[:8]):
    print(f"{a:>10,} | {p:>10,.0f} | {p-a:>+10,.0f}")

```

```

=====
MODEL EVALUATION REPORT
=====
R² Score : 0.9872 ✓ Excellent
MAE : Rs      13,439
RMSE : Rs     16,959
=====

Sample Predictions:
   Actual | Predicted | Error
-----
  489,899 |   478,320 | -11,579
  601,329 |   572,877 | -28,452
  623,788 |   650,489 | +26,701
  291,458 |   271,086 | -20,372
  390,040 |   344,431 | -45,609
  123,884 |   134,545 | +10,661
  484,442 |   443,295 | -41,147
  562,823 |   556,350 | -6,473

```

## Step 6 — Predict on New Houses

```

# New houses to price
new_houses = pd.DataFrame({
    'area_sqft': [1200, 2500, 800, 3500],
    'num_rooms': [3, 5, 1, 6 ],
    'age_years': [5, 10, 25, 2 ],
    'location_enc': [2, 1, 0, 2 ], # 2=Urban,1=Suburban,0=Rural
    'location_lbl': ['Urban', 'Suburban', 'Rural', 'Urban'],
})
predictions = model.predict(new_houses[features])
print("\n HOUSE PRICE PREDICTIONS")
print("=" * 60)
for i, (_, row) in enumerate(new_houses.iterrows()):
    print(f" House {i+1}: {row.area_sqft} sqft | "
          f"{row.num_rooms} rooms | "
          f"{row.age_years}yr old | "
          f"{row.location_lbl}")
    print(f" Estimated Price → Rs {predictions[i]:>10,.0f}")
    print()

```

```

HOUSE PRICE PREDICTIONS
=====
House 1: 1200 sqft | 3 rooms | 5yr old | Urban
Estimated Price → Rs      321,914

House 2: 2500 sqft | 5 rooms | 10yr old | Suburban
Estimated Price → Rs     534,588

House 3: 800 sqft | 1 rooms | 25yr old | Rural
Estimated Price → Rs     133,872

House 4: 3500 sqft | 6 rooms | 2yr old | Urban
Estimated Price → Rs     744,552

```



## Cheat sheat

## DATA PREPARATION

| Task             | Code                                  | Purpose                 |
|------------------|---------------------------------------|-------------------------|
| Import NumPy     | <code>import numpy as np</code>       | Numerical operations    |
| Import Pandas    | <code>import pandas as pd</code>      | Handle dataset (tables) |
| Create DataFrame | <code>df = pd.DataFrame({...})</code> | Build dataset           |

## Feature &amp; Target Selection Task

| Task         | Code                                  | Purpose         |
|--------------|---------------------------------------|-----------------|
| Features (X) | <code>X = df[['col1', 'col2']]</code> | Input variables |
| Target (y)   | <code>y = df['target']</code>         | Output variable |

## Train-Test Split

| Task         | Code                                                              | Purpose      |
|--------------|-------------------------------------------------------------------|--------------|
| Import split | <code>from sklearn.model_selection import train_test_split</code> | Split data   |
| Split data   | <code>train_test_split(X,y,test_size=0.2,random_state=42)</code>  | Train & test |

## Model Creation &amp; Training

| Task         | Code                                                           | Purpose          |
|--------------|----------------------------------------------------------------|------------------|
| Import model | <code>from sklearn.linear_model import LinearRegression</code> | Load algorithm   |
| Create model | <code>model = LinearRegression()</code>                        | Initialize model |
| Train model  | <code>model.fit(X_train, y_train)</code>                       | Learn patterns   |

## Prediction

| Task    | Code                                        | Purpose             |
|---------|---------------------------------------------|---------------------|
| Predict | <code>y_pred = model.predict(X_test)</code> | Predict unseen data |

## Model Evaluation

| Metric               | Code                                                     | Meaning                      |
|----------------------|----------------------------------------------------------|------------------------------|
| R <sup>2</sup> Score | <code>r2_score(y_test, y_pred)</code>                    | How well model explains data |
| MAE                  | <code>mean_absolute_error(y_test, y_pred)</code>         | Average error                |
| RMSE                 | <code>np.sqrt(mean_squared_error(y_test, y_pred))</code> | Penalizes large errors       |

## Model Parameters

| Task            | Code                          | Meaning                    |
|-----------------|-------------------------------|----------------------------|
| Slope (weights) | <code>model.coef_</code>      | Importance of each feature |
| Intercept       | <code>model.intercept_</code> | Base value                 |

## Data Encoding (Categorical → Numeric)

| Task           | Code                                     | Purpose                 |
|----------------|------------------------------------------|-------------------------|
| Label Encoding | <code>LabelEncoder()</code>              | Convert text to numbers |
| Fit transform  | <code>le.fit_transform(df['col'])</code> | Encode categories       |

## Data Reshaping

| Task           | Code                       | Meaning               |
|----------------|----------------------------|-----------------------|
| Reshape column | <code>reshape(-1,1)</code> | Convert 1D → 2D       |
| Auto rows      | <code>-1</code>            | Let NumPy decide rows |
| Single feature | <code>1</code>             | One column            |

## Noise

| Concept | Code                                      | Meaning              |
|---------|-------------------------------------------|----------------------|
| Noise   | <code>np.random.normal(0, 3000, n)</code> | Random variation     |
| Purpose | —                                         | Makes data realistic |

## Full Model Flow

| Step | Action                  |
|------|-------------------------|
| 1    | Create dataset          |
| 2    | Encode categorical data |
| 3    | Split data              |
| 4    | Train model             |

| Step | Action   |
|------|----------|
| 5    | Predict  |
| 6    | Evaluate |